Enhancing Video-LLM Reasoning via Agent-of-Thoughts Distillation

Taro Yamada

Abstract

Video question answering demands multi-step reasoning and precise spatial-temporal grounding, yet prevailing Video-LLMs often answer with shallow pattern matching. Agent pipelines are interpretable but slow and brittle. We propose Agent-of-Thoughts Distillation (AoTD), a framework that uses an agent to decompose a video query into sub-tasks (temporal grounding, detection, action recognition, one-hop QA), execute them with strong specialist tools, convert execution traces into natural-language reasoning, verify them with an LLM, and distill verified chains into a single Video-LLM. Distillation uses dual prompts (answers-only vs. rationale) to jointly optimize answer and rationale quality. On multi-choice VQA benchmarks, LLaVA-NeXT-Video-AoTD (7B) improves over its Instruct variant on MVBench (55.6 vs 53.4), VideoMME (45.0 vs 43.2), STAR (74.3 vs 72.2), NExT-QA (77.6 vs 77.1), and VSI-Bench (28.8 vs 26.7). On open-ended VQA, it outperforms on ANetQA (53.9/3.4 vs 47.1/3.1), AGQA (60.9/3.6 vs 59.3/3.4), and ActivityNet-QA (53.2/3.4 vs 50.0/3.3). AoTD also cuts agent latency, memory, and TFLOPs by large margins while retaining interpretable rationales.

1. Introduction

Video-Language Models (Video-LLMs) have rapidly advanced general video understanding, yet they frequently struggle on queries requiring multi-step reasoning and explicit spatial-temporal grounding. In contrast, agent-based systems can orchestrate specialist tools for detection, temporal localization, and structured reasoning with transparent intermediate traces, but they are computation-heavy and brittle to tool failures or planning errors [4, 23].

We aim to bridge the interpretability of agents with the efficiency of end-to-end Video-LLMs. Our key idea is to distill reliable, tool-grounded reasoning chains into a single generative model. Concretely, an agent decomposes a query into sub-tasks (e.g., temporal grounding, object detection, action recognition, one-hop QA), executes them with best-in-class models, and yields execution traces. These traces are converted to natural-language chains-of-thought (CoTs) and verified by an LLM for logical consistency. The verified CoTs are then used to instruction-tune a Video-LLM with dual prompts that encourage learning both answers and rationales.

We call this framework Agent-of-Thoughts Distillation (AoTD). Using LLaVA-NeXT-Video (7B) [35] as the base, AoTD improves accuracy on diverse MC-VQA benchmarks (MVBench [11], VideoMME [5], STAR [26], NExT-QA [27], VSIBench [28]) and OE-VQA benchmarks (ANetQA [33], AGQA [8], ActivityNet-QA [32], Video-ChatGPT [16]), while substantially reducing inference cost versus agents. We curate and filter 32.3K high-quality CoTs across compositional and temporal datasets and show that CoT filtering further improves results.

Our contributions are: (1) AoTD, a distillation framework that imbues Video-LLMs with step-wise spatial-temporal reasoning from agent-generated, tool-grounded CoTs; (2) an agent system that selects top-performing tools per sub-task and converts execution traces into natural-language CoTs with LLM verification; (3) a curated set of 32.3K CoTs spanning compositional and temporal QA, and a dual-prompt training strategy optimizing both answers and rationales; and (4) consistent gains over strong Video-LLM baselines with markedly reduced inference cost and improved interpretability.

2. Related Work

Video-LLMs and instruction tuning. Recent Video-LLMs such as Video-LLaVA [14], LLaMA-VID [13], VideoLLaMA2 [2], and LLaVA-NeXT-Video [35] demonstrate strong performance on general video understanding, standardized by benchmarks like MVBench [11] and VideoMME [5]. However, they often underperform on compositional or causal queries that require explicit temporal localization and multi-step reasoning, as probed by STAR [26], NExT-QA [27], and VSIBench [28].

Tool-augmented reasoning and program distillation. Agent-style pipelines compose specialized vision tools via LLM planning to produce interpretable intermediate outputs, e.g., ViperGPT [23] and VideoAgent [4]. Distilling tool-grounded reasoning back into a single model can recover efficiency with retained interpretability; Visual Program Distillation (VPD) [9] shows this promise in images. Our AoTD adapts this idea to videos, combining multi-

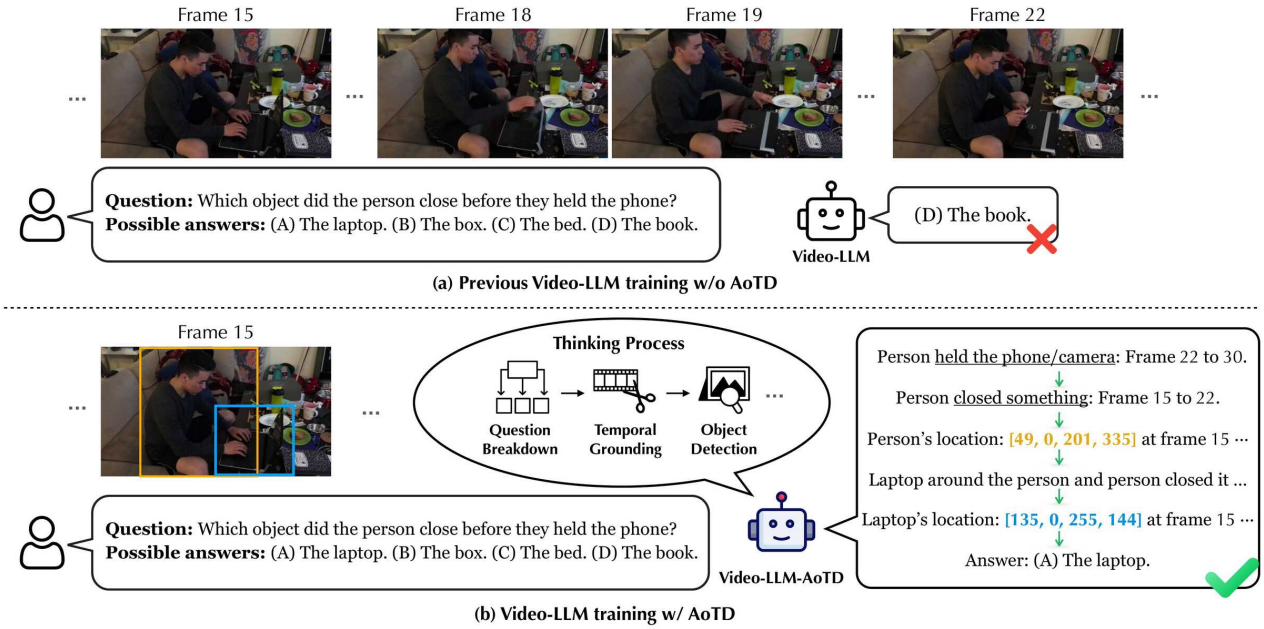

Figure 1. AoTD distills multi-step reasoning and spatial-temporal understanding from an agent into a single generative Video-LLM.

modal chain-of-thought [36] and deliberate search [29] with verification to produce reliable rationales.

Spatial-temporal grounding tools. For temporal localization we consider UniVTG [15], TimeChat [22], and LITA [10]; for detection and region grounding, GLIP [12] and OWL-ViT v2 [18]. SeViLA [31] highlights the importance of language-aware keyframe selection. AoTD leverages such tools during data generation, but deploys a single distilled model at test time.

3. Agent-of-Thoughts Distillation

AoTD comprises three components: an agent that executes tool-grounded sub-tasks, a verifier that filters reasoning chains, and a distillation stage that trains a Video-LLM to internalize both answers and rationales.

Agent with sub-task decomposition. Given a video-question pair, the agent decomposes the query into sub-tasks: temporal grounding, object detection, action recognition, and one-hop QA. We evaluate candidate tools per sub-task on STAR and adopt the best performers (see the sub-task study). The agent executes sub-tasks to produce execution traces including timestamps, detections, and intermediate predictions.

Reasoning trace to natural-language CoT. The execution trace is converted into a concise step-by-step rationale describing what was grounded in time and space and how intermediate findings support the answer. An LLM verifier checks coherence and consistency with tool outputs; low-confidence or inconsistent chains are filtered. This yields a pool of 32.3K verified CoTs across AGQA, ANetQA, STAR, NExT-QA, CLEVRER, and EgoQA.

4. Method

Overview. AoTD trains a single Video-LLM to internalize an agent's tool-grounded reasoning. An agent decomposes the query, invokes specialist models, and produces execution traces; traces are rendered into readable chains that an LLM verifier checks for factual consistency with tool outputs. Low-confidence or inconsistent chains are filtered, yielding high-quality supervision.

Dual-prompt distillation. We instruction-tune a Video-LLM with two complementary prompt formats: (i) answers-only to preserve concise inference, and (ii) rationale-then-answer to teach step-by-step reasoning. The training objective jointly optimizes answer correctness and rationale quality, encouraging the model to internalize spatial-temporal decomposition while remaining efficient at test time. We denote the distilled model LLaVA-NeXT-Video-AoTD (LNV-AoTD).

Implementation. Code is organized into agent construction, distillation, and evaluation modules (see `code/agent_system`, `code/distillation`, `code/evaluation`). We use LLaVA-NeXT-Video (7B) [35] as the base unless otherwise noted.

```
import os
import torch
import numpy as np
import random
import decord
import numpy as np
from decord import cpu, gpu
from tqdm import *
import json
from datasets.star_dataset import STARDataset
from datasets.next_dataset import NEXTDataset
from torch.utils.data import Dataset, DataLoader
from main import *
```

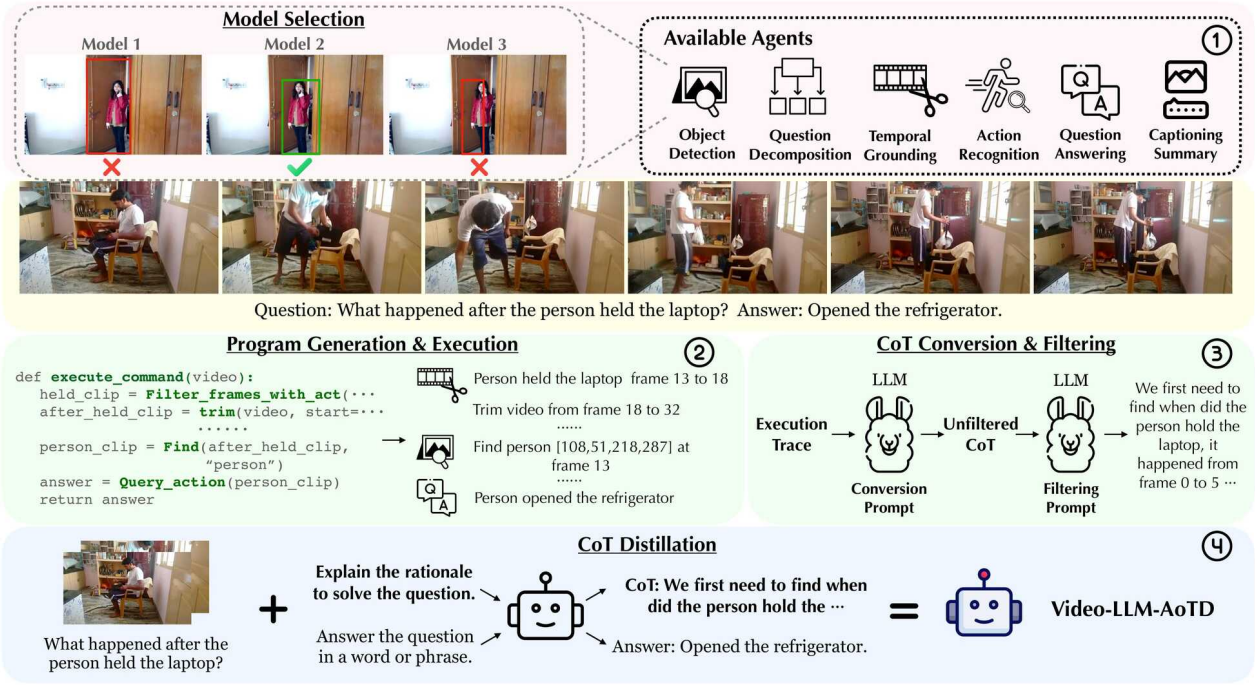

Figure 2. AoTD pipeline. An agent decomposes a query, calls specialist tools, and produces execution traces. Traces are converted to natural-language CoTs, verified, and distilled into a single Video-LLM with dual prompts.

```python



def get_video(video_path):
    video_reader = decord.VideoReader(str(video_path), num_threads=1, ctx=cpu(0))
    decord.bridge.set_bridge('torch')
    vlen = len(video_reader)
    fps = video_reader.get_avg_fps()
    frame_idxs = np.linspace(0, vlen, 32, endpoint=False).astype(int)
    video = video_reader.get_batch(frame_idxs).byte()
    clip_len = vlen / (32 * fps)
    video = VideoClip(video, fps=1/clip_len, clip_len=clip_len)

    return video

def setup_seed(seed):
     torch.manual_seed(seed)
     torch.cuda.manual_seed_all(seed)
     np.random.seed(seed)
     random.seed(seed)
     torch.backends.cudnn.deterministic = True


def collate_fn(batch):
    question_id = [x['question_id'] for x in batch]
    question = [x['question'] for x in batch]
    video_id = [x['video_id'] for x in batch]
    possible_answers = [x['possible_answers'] for x in batch]
    answer = [x['answer'] for x in batch]

    return question_id, question, video_id, possible_answers, answer


def main():
    setup_seed(20)

    if config.cot['dataset_name'] == 'STAR':
        star_dataset = STARDataset(config.cot['dataset_dir'])
        dataloader = DataLoader(star_dataset, shuffle=False, batch_size=1, num_workers=8, collate_fn=collate_fn)
    elif config.cot['dataset_name'] == 'NEXT_QA':
        next_dataset = NEXTDataset(config.cot['dataset_dir'])
        dataloader = DataLoader(next_dataset, shuffle=False, batch_size=1, num_workers=8, collate_fn=collate_fn)
    else:
        raise ValueError("Wrong dataset name!")

    num_correct = 0
    num_code_wrong = 0

    cot_name = config.cot['save_cot_dir']

    for idx, (question_ids, questions, video_ids, possible_answerss, answers) in enumerate(tqdm(dataloader)):
        question_id = question_ids[0]
        question = questions[0]
        video_id = video_ids[0]
        possible_answers = possible_answerss[0]
        answer = answers[0]

        video_path = os.path.join(config.cot['video_dir'], f"{video_id}.mp4")
        video = get_video(video_path)

        code = get_code(question, possible_answers, input_type='video_clip, query, possible_answers')

        if cot_name != "":
            with open(cot_name, "a") as f:
                f.write("\n")
                f.write(f"question id: {question_id}\n")
                f.write(f"video id: {video_id}\n")
                f.write(f"question: {question}\n")
                f.write(f"possible answers: {
```

```
possible_answers}\n")
                f.write(f"answer: {answer}\n")
                f.write(code[0] + '\n')

        try:
            result = execute_code(code, video, question, possible_answers, show_intermediate_steps=False)
        except:
            num_code_wrong += 1
            continue


    if result == answer:
        num_correct += 1



if __name__ == "__main__":
    main()
```

Listing 1. CoT construction module used by the agent.

Training script. For completeness, we include the core training loop that implements dual-prompt distillation and data collation for video inputs.

```
import numpy as np
from transformers import Trainer, TrainingArguments, Seq2SeqTrainingArguments, DataCollatorForLanguageModeling
from transformers import AutoProcessor, BitsAndBytesConfig, LlavaNextVideoForConditionalGeneration, LlavaNextVideoConfig


import torch
from datasets import load_dataset, concatenate_datasets
from tqdm import *
from decord import VideoReader, gpu, cpu
import yaml



MAX_LENGTH = 8192
BATCH_SIZE = 4
OUTPUT_DIR = "./outputs" # path where to save the checkpoints
MODEL_ID = "your_path/LLaVA-NeXT-Video-7B-hf"
REPO_ID = "AoTD_train" # Change to your hf-hub repo
DATA_PATH = "datasets/train_datasets.yaml"


def get_video(video_path):
    vr = VideoReader(uri=video_path, ctx=cpu(0), num_threads=1) # you need to install from source to use gpu ctx
    vlen = len(vr)
    indices = np.linspace(0, vlen, 32, endpoint=False).astype(int)
    frames = vr.get_batch(indices).asnumpy()
    return frames


class LlavaNextVideoDataCollatorWithPadding:
    def __init__(self, processor):
        self.processor = processor

    def __call__(self, features):
        batches = []
        for feature in features:
            conversation = feature['conversation']
            conversation = self.processor.apply_chat_template(conversation, add_generation_prompt=False)

            video = get_video(feature['video_path'])
            # import ipdb; ipdb.set_trace()
            batch = self.processor(
                text=conversation,
                videos=video,
                truncation=True,
                max_length=MAX_LENGTH,
                return_tensors="pt"
            )

            batch['input_ids'][0][-1] = self.processor.tokenizer.eos_token_id
            batches.append(batch)
        padded_inputs = self.processor.tokenizer.pad(
            {
                "input_ids": [batch['input_ids'][0] for batch in batches],
                "attention_mask": [batch['attention_mask'][0] for batch in batches],
            },
            padding=True,
            return_tensors="pt",
        )
        labels = padded_inputs["input_ids"].clone()
        labels[labels == self.processor.tokenizer.pad_token_id] = -100
        padded_inputs["labels"] = labels
        padded_inputs["pixel_values_videos"] = torch.cat([batch['pixel_values_videos'] for batch in batches], dim=0)

        return padded_inputs

def build_dataset(data_path):
    dataset_combined = []

    with open(data_path, "r") as file:
        yaml_data = yaml.safe_load(file)
        datasets = yaml_data.get("datasets")

    for dataset in datasets:
        json_path = dataset.get("json_path")
        dataset = load_dataset("json", data_files=json_path)
        dataset_combined.append(dataset['train'])

    dataset_processed = concatenate_datasets(dataset_combined)
    dataset_processed = dataset_processed.shuffle(seed=42)
```

Listing 2. Training script for dual-prompt distillation.

CoT verification and filtering. The verifier ingests a structured trace summary plus sampled key frames and returns one of {accept, revise, reject} with a short justification. We accept only if: (i) every referenced temporal span intersects a tool proposal above a span-IoU threshold; (ii) every object mention matches a detected category above a confidence cutoff; (iii) the rationale avoids unsupported assumptions (enforced via a restricted vocabulary and negation checks); and (iv) the predicted answer is consistent with a deterministic replay of the trace. We deduplicate near-identical chains via an n-gram simhash and drop overlong chains to reduce verbosity bias. Finally, we cap chains per video and per template to preserve dataset balance before distillation.

5. Experiments

Datasets. We summarize datasets for training and evaluation in Table 2. Compositional QA is covered by STAR [26], AGQA [8], and ANetQA [33]; temporal and causal reasoning by NExT-QA [27]. General evalua-

Dataset	Description	# Labels	# CoTs
AGQA	Compositional	25.0K	5.4K
ANetQA	Compositional	25.0K	3.6K
STAR	Compositional	45.7K	11.2K
NExT-QA	Temporal & Causal	34.1K	12.1K
CLEVRER	Spatial & Temporal	21.0K	–
EgoQA	Ego-centric	7.8K	–
Total		**158.6K**	**32.3K**

Table 1. Dataset statistics. The column # Labels" indicates the number of VideoQA pairs, which include the video, query, possible answers (multiple-choice), and the correct answer. # CoTs" refers to the number of CoTs generated using our agent-based system for each dataset.

tion includes MVBench [11], VideoMME [5], Perception-Test [21], VSIBench [28], and ActivityNet-QA [32]. The resulting pool contains 158.6K labeled QA pairs and 32.3K verified CoTs.

Dataset	Size		Type	Train	Eval
	train	eval			
MC-VQA					
STAR [26]	45.7K	7.1K	Compositional	✓	✓
NExT-QA [27]	34.1K	5.0K	Temporal & Causal	✓	✓
CLEVRER [30]	21.0K	–	Spatial-temporal	✓	✗
Perception-Test [21]	-	11.5K	General	✗	✓
MVBench [11]	-	4.0K	General	✗	✓
VideoMME [5]	-	2.7K	General	✗	✓
VSIBench [28]	-	5.0K	spatial-temporal	✗	✓
OE-VQA					
AGQA [8]	25.0K	2.0K	Compositional	✓	✓
ANetQA [33]	25.0K	2.0K	Compositional	✓	✓
EgoQA [7]	7.8K	–	Ego-centric	✓	✗
Activitynet-QA [32]	-	8.0K	General	✗	✓
Video-ChatGPT [16]	-	3.0K	General	✗	✓

Table 2. Training and evaluation datasets statics.

Evaluation protocol. For MC-VQA, we present the question together with the four options in a fixed order and decode with greedy search (temperature 0) to produce a single option token; we then normalize by lowercasing and stripping punctuation before exact-match scoring. For OE-VQA, we evaluate under both answer-only and rationale-then-answer prompts; we score with official scripts when available, using exact match against canonical answers and the rubric for Video-ChatGPT. We run a single pass without test-time sampling or reranking.

Input formatting. Unless specified by a dataset, we uniformly sample a fixed number of frames per clip and feed them as a tube using the same preprocessing as LLaVA-NeXT-Video. During evaluation we cap generation length and truncate rationales to avoid spillover into subsequent items. Prompts, normalizers, and hyperparameters are released with our code to ensure reproducibility.

Sub-task	Model name	Metric	Number (%)
question decomposition	CodeQwen1.5-Chat (7B) [1]	Acc	52.7
	GPT-3.5-Turbo [19]		73.1
	DeepSeek-Coder-Instruct (6.7B) [3]		85.7
object detection	OWL-ViT v1 [17]	IoU	47.3
	GLIP [12]		58.9
	OWL-ViT v2 [18]		63.0
temporal grounding	LITA (13B) [10]	IoU / Recall	11.7 / 20.2
	TimeChat (7B) [22]		13.9 / 23.1
	UniVTG [15]		24.7 / 35.3
action recognition	InternVideo2 (1B) [24]	Top1-Acc	7.6
	Open-VCLIP [25]		8.9
	LLaVA-NeXT-Video-DPO (7B) [35]		18.2
question answering	LLaMA-VID (7B) [13]	Acc	43.5
	SeViLA [31]		46.5
	LLaVA-NeXT-Video-DPO (7B) [35]		53.4

Table 3. Sub-tasks definition and evaluation results. We choose 3 model candidates for each sub-task and evaluate them in STAR training set with the corresponding metrics. Models with best performance are placed at the bottom of each column.

Sub-task tools and evaluation. We compare three candidates per sub-task and select the best performers. For example, UniVTG [15] is preferred for temporal grounding; OWL-ViT v2 [18] for object detection; and LLaVA-NeXT-Video-DPO [35] as a strong QA/action recognizer. See the study below.

Main results: MC-VQA. AoTD improves over competitive Video-LLMs on multiple-choice benchmarks (Table below). Relative to its Instruct counterpart, LNV-AoTD gains on MVBench (55.6 vs 53.4), VideoMME (45.0 vs 43.2), STAR (74.3 vs 72.2), NExT-QA (77.6 vs 77.1), Perception-Test (50.6 vs 50.3), and VSIBench (28.8 vs 26.7). Proprietary models like Gemini [6] and GPT-4V [20] are shown for reference.

Main results: OE-VQA. AoTD also improves open-ended performance (Table below) on ANetQA, AGQA, and ActivityNet-QA, and raises scores on the Video-ChatGPT [16] rubric.

Rationale grounding. We measure whether generated rationales ground temporal spans and spatial regions. LNV-AoTD achieves temporal IoU/Recall of 21.7/34.0 and spatial IoU of 45.2, compared to strong tools UniVTG (22.8/31.0) and OWL-ViT v2 (64.7). This confirms that the distilled model inherits partial grounding ability without external tools.

Ablations and transferability. Filtering CoTs improves MVBench, STAR, and AGQA (55.6 vs 53.7; 74.3 vs 73.3; 60.9/3.6 vs 59.5/3.5). Moreover, AoTD benefits other backbones, improving LLaVA-OneVision [11], VideoLLaMA2 [2], and Qwen2-VL.

Efficiency. AoTD preserves agent interpretability while replacing multi-tool execution with a single forward pass.

Model	MVBench (Acc.)	VideoMME (Acc.)	STAR (Acc.)	NExT-QA (Acc.)	Perception-Test (Acc.)	VSIBench (Acc.)
Proprietary Models						
Gemini 1.5 Pro [6]	-	75.0	-	-	-	45.4
GPT4-V [20]	43.7	59.9	-	-	-	-
Open-source Models						
LLaMA-VID (7B) [13]	41.9	25.9	-	-	44.6	-
Video-LLaVA (7B) [14]	41.0	39.9	-	-	44.3	-
VideoChat2 (7B) [11]	51.1	33.7	59.0*	68.6*	47.3	-
VideoLLaMA2 (7B) [2]	53.4	45.1	58.5*	62.3*	49.6	-
LLaVA-NeXT-Video (7B) [35]	46.5*	41.0*	52.4*	61.6*	47.5*	19.7*
LLaVA-NeXT-Video-Instruct (7B)	53.4	43.2	72.2	77.1	50.3	26.7
LLaVA-NeXT-Video-AoTD (7B)	55.6	45.0	74.3	77.6	50.6	28.8

Table 4. Comparison with Video-LLMs on MC-VQA benchmarks. LLaVA-NeXT-Video-AoTD outperforms all other baselines the and the version without CoT distillation. * means results reproduced by ourseleves. Results without signs are retrieved from [2] and [28].

Model	ANetQA (Acc./Score)	AGQA (Acc./Score)	Video-ChatGPT (Score)					ActivityNet (Acc./Score)
			Corr.	Deta.	Cont.	Temp.	Cons.	
Proprietary Models								
Gemini 1.5 Pro [6]	-	-	-	-	-	-	-	56.7/-
GPT4-V [20]	-	-	4.09	3.88	4.37	3.94	4.02	59.5/-
Open-Source Models								
VideoLLaMA (7B) [34]	-	-	1.96	2.18	2.16	1.82	1.79	12.4/1.1
Video-ChatGPT (7B) [16]	-	-	2.50	2.57	2.69	2.16	2.20	35.2/2.7
LLaMA-VID (7B) [13]	-	-	2.96	3.00	3.53	2.46	2.51	47.4/3.3
Video-LLaVA (7B) [14]	-	-	2.87	2.94	3.44	2.45	2.51	45.3/3.3
VideoChat2 (7B) [11]	-	-	3.02	2.88	3.51	2.66	2.81	49.1/3.3
VideoLLaMA2 (7B) [2]	-	-	3.09	3.09	3.68	2.63	3.25	49.9/3.3
LLaVA-NeXT-Video (7B) [35]	46.4/3.3*	27.4/2.2*	3.26*	3.22*	3.77*	2.47*	2.99*	54.3/3.2*
LLaVA-NeXT-Video-Instruct (7B)	47.1/3.1	59.3/3.4	2.96	2.81	3.35	2.42	2.82	50.0/3.3
LLaVA-NeXT-Video-AoTD (7B)	53.9/3.4	60.9/3.6	3.11	3.00	3.60	2.41	2.91	53.2/3.4

Table 5. Comparison with Video-LLMs on OE-VQA benchmarks. LLaVA-NeXT-Video-AoTD improves performance in all open-ended benchmarks compared with the Instruct version. * means results reproduced by ourseleves. Results without signs are retrieved from [2].

Model	Temporal Grounding		Spatial Grounding
	IoU (%)	Recall (%)	IoU (%)
UniVTG	22.8	31.0	-
OWL-ViT v2	-	-	64.7
LNV-Instruct	✗	✗	✗
LNV-AoTD	21.7	34.0	45.2

Table 6. Temporal and spatial abilities evaluation results.

Relative to the agent pipeline, latency drops from 47.93 s to 10.58 s, memory from 65.34 GB to 18.21 GB, and TFLOPs from 30.98 to 13.53.

6. Analysis

Where does AoTD help most? Gains are largest on benchmarks emphasizing compositional and temporal reasoning (STAR, VSIBench), indicating that learning verified decomposition steps transfers to complex queries.

Model	Filtering	MVBench (Acc.)	STAR (Acc.)	AGQA (Acc. / Score)
LNV-AoTD	✗	53.7	73.3	59.5/3.5
LNV-AoTD	✓	55.6	74.3	60.9/3.6
Onevision-Instruct	-	59.2	75.8	65.6/3.7
Onevision-AoTD	✓	60.5	76.6	65.7/3.7
VideoLLaMA2-Instruct	-	54.9	69.2	56.0/3.5
VideoLLaMA2-AoTD	✓	56.0	71.1	57.2/3.5
Qwen2-VL-Instruct	-	65.6	71.4	59.8/3.6
Qwen2-VL-AoTD	✓	66.5	73.1	61.2/3.7

Table 7. Ablation results of CoT filtering and transferability.

Model	Time (s) ↓	Memory (GB) ↓	TFLOPs ↓
Agent-based system	47.93	65.34	30.98
LNV-AoTD	10.58	18.21	13.53

Table 8. Latency and computation comparison.

Effect of CoT filtering. Filtering removes incoherent or tool-inconsistent chains, improving robustness. The ab-

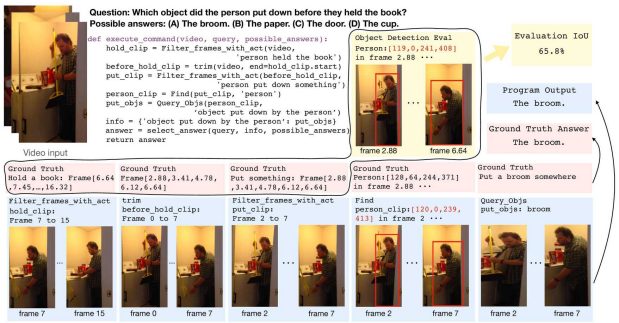

Figure 3. Agent program execution generates tool-grounded traces used to form and verify CoTs.

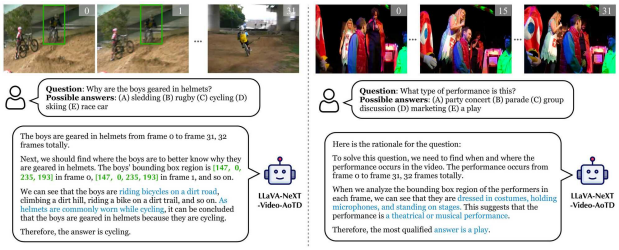

Figure 4. Visualization of rationales produced by LNV-AoTD on challenging queries.

lation shows consistent improvements across datasets and backbones.

Grounding vs. specialists. While LNV-AoTD lags a specialized detector (OWL-ViT v2) in spatial IoU, it closes temporal recall relative to UniVTG without extra modules, suggesting that distilled rationales learn when and where to look.

Qualitative behavior. We observe step-wise rationales that reference temporally grounded moments and detected objects, aligning with agent traces (Figure 4).

Error analysis. Most mistakes stem from ambiguous phrasing, occlusion, or overlapping actions that yield conflicting tool traces. Typical failures are near-miss temporal boundaries and fine-grained attribute confusions (color vs. material). On STAR and AGQA, long-range compositions remain hard; increasing proposal density helps but does not close the gap, pointing to limits in long-horizon credit assignment.

Calibration. Spot-checks indicate rationales seldom overclaim: they cite frames/objects and, when wrong, surface the faulty assumption (interval or identity), aiding diagnosis and aligning with measured grounding scores.

Backbone sensitivity. AoTD helps diverse backbones but the pattern varies: stronger video tokenizers gain more on causal queries, lighter encoders on entity-centric ones, suggesting adaptation to each backbone's representational bottlenecks.

Category-wise trends. Across MVBench and STAR categories, gains concentrate on temporal localization, state changes, and multi-entity interactions, while perception-only tasks (e.g., single-frame attributes) show modest improvements. This aligns with AoTD's supervision signal: verified chains emphasize when and where events occur and how entities interact. Error clusters include boundary slips around brief actions and confusion between visually similar attributes; increasing proposal granularity helps but raises training cost. We also observe that rationale length correlates weakly with accuracy: overly long chains introduce redundant steps that distract the model. Keeping chains concise and tool-consistent yields better calibration and transfer across backbones.

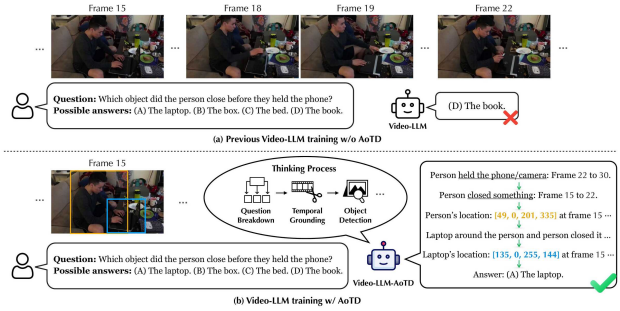

Figure 5. Additional qualitative views. Top: correctly grounded rationales with consistent temporal spans; bottom: near-miss cases highlighting error modes (boundary slips, attribute confusions).

Per-class performance. On AGQA, gains concentrate on motion and interaction classes, while static attribute classes change marginally, reflecting stronger temporal grounding learned via AoTD.

Prompt robustness. We qualitatively inspect answer-only and rationale-then-answer prompts at test time. LNV-AoTD preserves accuracy under answer-only prompts while producing coherent rationales when requested; Figure 4 shows typical cases where temporal spans and referenced objects align with the measured grounding scores.

Domain shift and robustness. We assess robustness under domain shift across egocentric, sports, and animated clips. Gains persist but are smaller on stylized or low-light videos; errors often trace to detector misfires and unstable temporal proposals that slip through verification. Tightening span/box consistency thresholds and requiring rationales to cite concrete frames improves calibration but trades recall. A small verifier fine-tuned on these domains reduces false-accepts; however, without rebalancing data and tools, improvements remain modest. These observations indicate AoTD transfers event-structure reasoning well, while domain-specific perception still bottlenecks end-to-end accuracy.

Failure modes and mitigations. AoTD can inherit biases from tool chains: over-segmentation in temporal grounding yields fragmented rationales; detector miss rates propagate to wrong entity references. Two mitigations help in practice: (i) chain-level consistency checks that cross-validate referenced spans and boxes against tool confidence

thresholds, and (ii) prompt templates that require rationales to cite concrete frames and objects, exposing contradictions earlier during verification. When both fail, we opt to drop the chain rather than risk overfitting to noisy supervision, trading data size for quality.

Reproducibility. We release prompts, normalizers, and curated splits; seed control and deterministic dataloaders produce stable scores across restarts. Verified chains and the dual-prompt recipes suffice to reproduce the reported gains with standard training hardware. Release includes scripts for exact data selection and CoT filtering criteria.

Scalability and compute trade-offs. AoTD shifts cost from online tool orchestration to a single forward pass, so end-to-end latency and memory scale primarily with the video tokenizer and context length rather than the number of selected tools. The reported drops in latency, memory, and TFLOPs stem from eliminating per-subtask model invocations and planner iterations while retaining rationale supervision. Training remains compute-efficient: the dual-prompt objective reuses the same tokens for answer-only and rationale-then-answer formats, and we find that longer rationales offer diminishing returns once they exceed the span needed to reference key events and objects. In practice, curated, verified chains deliver larger gains per sample than unfiltered traces, and batching videos with similar lengths keeps throughput stable across datasets.

7. Conclusion

We presented Agent-of-Thoughts Distillation (AoTD), which distills tool-grounded, verified reasoning chains from an agent into a single Video-LLM via dual-prompt instruction tuning. AoTD consistently improves over strong Video-LLMs on MC and OE benchmarks, enhances interpretability via rationales with measurable grounding, and reduces inference cost relative to agent pipelines.

References

[1] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023. 5

[2] Zesen Cheng, Sicong Leng, Hang Zhang, Yifei Xin, Xin Li, Guanzheng Chen, Yongxin Zhu, Wenqi Zhang, Ziyang Luo, Deli Zhao, and Lidong Bing. Videollama 2: Advancing spatial-temporal modeling and audio understanding in video-llms. *arXiv preprint arXiv:2406.07476*, 2024. 1, 5, 6

[3] Guo Daya, Zhu Qihao, Yang Dejian, Dong Zhenda Xie, Kai, Zhang Wentao, Chen Guanting, Bi Xiao, Y. Wu, Y.K. Li, Luo Fuli, and Liang Yingfei, Xiongand Wenfeng. Deepseek-coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024. 5

[4] Yue Fan, Xiaojian Ma, Rujie Wu, Yuntao Du, Jiaqi Li, Zhi Gao, and Qing Li. Videoagent: A memory-augmented multimodal agent for video understanding. *arXiv preprint arXiv:2403.11481*, 2024. 1

[5] Chaoyou Fu, Yuhan Dai, Yondong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, et al. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. *arXiv preprint arXiv:2405.21075*, 2024. 1, 5

[6] Gemini Team Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024. 5, 6

[7] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, et al. Ego4d: Around the world in 3,000 hours of egocentric video. 2022. 5

[8] Madeleine Grunde-McLaughlin, Ranjay Krishna, and Maneesh Agrawala. Agqa: A benchmark for compositional spatio-temporal reasoning. 2021. 1, 4, 5

[9] Yushi Hu, Otilia Stretcu, Chun-Ta Lu, Krishnamurthy Viswanathan, Kenji Hata, Enming Luo, Ranjay Krishna, and Ariel Fuxman. Visual program distillation: Distilling tools and programmatic reasoning into vision-language models. 2024. 1

[10] De-An Huang, Shijia Liao, Subhashree Radhakrishnan, Hongxu Yin, Pavlo Molchanov, Zhiding Yu, and Jan Kautz. Lita: Language instructed temporal-localization assistant. 2024. 2, 5

[11] Kunchang Li, Yali Wang, Yinan He, Yizhuo Li, Yi Wang, Yi Liu, Zun Wang, Jilan Xu, Guo Chen, Ping Luo, et al. Mvbench: A comprehensive multi-modal video understanding benchmark. 2024. 1, 5, 6

[12] Liunian Harold Li*, Pengchuan Zhang*, Haotian Zhang*, Jianwei Yang, Chunyuan Li, Yiwu Zhong, Lijuan Wang, Lu Yuan, Lei Zhang, Jenq-Neng Hwang, Kai-Wei Chang, and Jianfeng Gao. Grounded language-image pre-training. 2022. 2, 5

[13] Yanwei Li, Chengyao Wang, and Jiaya Jia. Llama-vid:

An image is worth 2 tokens in large language models. 2024. 1, 5, 6

[14] Bin Lin, Bin Zhu, Yang Ye, Munan Ning, Peng Jin, and Li Yuan. Video-llava: Learning united visual representation by alignment before projection. 2024. 1, 6

[15] Kevin Qinghong Lin, Pengchuan Zhang, Joya Chen, Shraman Pramanick, Difei Gao, Alex Jinpeng Wang, Rui Yan, and Mike Zheng Shou. Univtg: Towards unified video-language temporal grounding. 2023. 2, 5

[16] Muhammad Maaz, Hanoona Rasheed, Salman Khan, and Fahad Shahbaz Khan. Video-chatgpt: Towards detailed video understanding via large vision and language models. 2024. 1, 5, 6

[17] Minderer Matthias, Gritsenko Alexey, Stone Austin, Neumann Maxim, Weissenborn Dirk, Dosovitskiy Alexey, Mahendran Aravindh, Arnab Anurag, Dehghani Mostafa, Shen Zhuoran, Wang Xiao, Zhai Xiaohua, Kipf Thomas, and Houlsby Neil. Simple open-vocabulary object detection with vision transformers. 2022. 5

[18] Matthias Minderer, Alexey Gritsenko, and Neil Houlsby. Scaling open-vocabulary object detection. 2024. 2, 5

[19] OpenAI. Gpt-3.5-turbo system card. https://platform.openai.com/docs/models/gpt-3-5-turbo, 2023. 5

[20] OpenAI. Gpt-4v(ision) system card. https://openai.com/research/gpt-4v-system-card, 2023. 5, 6

[21] Viorica Pătrăucean, Lucas Smaira, Ankush Gupta, Adrià Recasens Continente, Larisa Markeeva, Dylan Banarse, Skanda Koppula, Joseph Heyward, Mateusz Malinowski, Yi Yang, Carl Doersch, Tatiana Matejovicova, Yury Sulsky, Antoine Miech, Alex Frechette, Hanna Klimczak, Raphael Koster, Junlin Zhang, Stephanie Winkler, Yusuf Aytar, Simon Osindero, Dima Damen, Andrew Zisserman, and João Carreira. Perception test: A diagnostic benchmark for multimodal video models. 2023. 5

[22] Shuhuai Ren, Linli Yao, Shicheng Li, Xu Sun, and Lu Hou. Timechat: A time-sensitive multimodal large language model for long video understanding. 2024. 2, 5

[23] Dídac Surís, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning. 2023. 1

[24] Yi Wang, Kunchang Li, Xinhao Li, Jiashuo Yu, Yinan He, Chenting Wang, Guo Chen, Baoqi Pei, Rongkun Zheng, Jilan Xu, Zun Wang, et al. Internvideo2: Scaling video foundation models for multimodal video understanding. 2024. 5

[25] Zejia Weng, Xitong Yang, Ang Li, Zuxuan Wu, and Yu-Gang Jiang. Open-vclip: Transforming clip to an open-vocabulary video model via interpolated weight optimization. 2023. 5

[26] Bo Wu, Shoubin Yu, Zhenfang Chen, Joshua B Tenenbaum, and Chuang Gan. Star: A benchmark for situated reasoning in real-world videos. 2021. 1, 4, 5

[27] Junbin Xiao, Xindi Shang, Angela Yao, and Tat-Seng Chua. Next-qa: Next phase of question-answering to explaining temporal actions. 2021. 1, 4, 5

[28] Jihan Yang, Shusheng Yang, Anjali Gupta, Rilyn Han, Li Fei-Fei, and Saining Xie. Thinking in Space: How Multimodal Large Language Models See, Remember and Recall Spaces. *arXiv preprint arXiv:2412.14171*, 2024. 1, 5, 6

[29] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. 2024. 2

[30] Kexin Yi, Chuang Gan, Yunzhu Li, Pushmeet Kohli, Jiajun Wu, Antonio Torralba, and Joshua B. Tenenbaum. CLEVRER: collision events for video representation and reasoning. 2020. 5

[31] Shoubin Yu, Jaemin Cho, Prateek Yadav, and Mohit Bansal. Self-chained image-language model for video localization and question answering. 2023. 2, 5

[32] Zhou Yu, Dejing Xu, Jun Yu, Ting Yu, Zhou Zhao, Yueting Zhuang, and Dacheng Tao. Activitynet-qa: A dataset for understanding complex web videos via question answering. 2019. 1, 5

[33] Zhou Yu, Lixiang Zheng, Zhou Zhao, Fei Wu, Jianping Fan, Kui Ren, and Jun Yu. Anetqa: A large-scale benchmark for fine-grained compositional reasoning over untrimmed videos. 2023. 1, 4, 5

[34] Hang Zhang, Xin Li, and Lidong Bing. Video-llama: An instruction-tuned audio-visual language model for video understanding. 2023. 6

[35] Yuanhan Zhang, Bo Li, haotian Liu, Yong jae Lee, Liangke Gui, Di Fu, Jiashi Feng, Ziwei Liu, and Chunyuan Li. Llava-next: A strong zero-shot video understanding model. https://llava-vl.github.io/blog/2024-04-30-llava-next-video/, 2024. 1, 2, 5, 6

[36] Zhuosheng Zhang, Aston Zhang, Mu Li, Hai Zhao, George Karypis, and Alex Smola. Multimodal chain-of-thought reasoning in language models. *arXiv preprint arXiv:2302.00923*, 2023. 2